

theorem

Vera: Weakest Preconditions for TICL over Rust

From Imp Structural Lemmas to Rust VCGen Rules

Lef Ioannidis

2026

Background: TICL over Imp

TICL Structural Lemmas for Imp

Core idea (from the TICL paper)

For a small imperative language `StImp` with mutable state, the entailment $[t, m \Vdash_L \varphi]_S$ is proved structurally via inference rules on the syntax of t .

Key rules from `imp-lemmas.tex`:

- **Seq**_SAU_L: sequential composition decomposes into bind
- **If**_SAU_L: conditional branches both satisfy the obligation
- **While**_SAG: infinite loop with coinductive invariant $\mathcal{R}$
- **While**_SAU_L: terminating loop with ranking function f

Goal: Lift these to Rust syntax $\rightarrow$ define $\text{wp}(_, _)$ for Vera.

wp(Rust, TIDL) — Weakest Preconditions

Notation

$\sigma \in \text{State}$ (mutable references, heap)

$[t, \sigma \Vdash_L \varphi]_{\text{Rust}} \triangleq \text{“Rust term } t \text{ from state } \sigma \text{ satisfies } \varphi\text{”}$

$\text{wp}(t, \varphi) \triangleq \text{weakest precondition on } \sigma \text{ s.t. } [t, \sigma \Vdash_L \varphi]_{\text{Rust}}$

Temporal formulas

- $\varphi ::= \text{now } P \mid \text{AG } \varphi \mid \varphi \text{ AU } \varphi' \mid \varphi \wedge \varphi'$
- Sugar: $\text{AF } \varphi' = \top \text{ AU } \varphi', \quad \text{AX } \varphi' = \top \text{ AN } \varphi'$
- done Q : termination postcondition; now P : intermediate state predicate

Rule: Assignment

Rust: $*x = e$

$$\frac{\sigma' = \sigma[x \mapsto v] \quad \varphi(\sigma') \quad \varphi'(\sigma') \vee (\text{continue checking})}{[*x = e, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{ASSIGN}_{\text{RUST}} \text{AU}_L$$

Weakest precondition

$$\text{wp}(*x = e, \varphi \text{ AU } \varphi') = \varphi[\sigma'/\sigma] \wedge (\varphi'[\sigma'/\sigma] \vee \top)$$

where $\sigma' = \sigma[x \mapsto \llbracket e \rrbracket_\sigma]$

Analogous to **Assign** in Imp: $\llbracket s \leftarrow x \rrbracket_s$ updates the state map.

Rule: Sequential Composition (let-binding)

Rust: `let x = t; u`

$$\frac{[t, \sigma \Vdash_R \varphi \text{ AU AX done } \mathcal{R}]_{\text{Rust}} \quad \forall x, \sigma', \mathcal{R} \ x \ \sigma' \rightarrow [u[x], \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}}{[\text{let } x = t; u, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{LET}_{\text{RUST}}\text{AU}_L$$

Weakest precondition

$$\text{wp}(\text{let } x = t; u, \varphi \text{ AU } \varphi') = \text{wp}(t, \varphi \text{ AU AX done } \mathcal{R}) \wedge \forall x, \sigma'. \mathcal{R} \ x \ \sigma' \rightarrow \text{wp}(u[x], \varphi \text{ AU } \varphi')$$

Direct lift of **BindAU_L** / **Seq_SAU_L** from TACL.

Rule: Conditional

Rust: `if c {t} else {u}`

$$\frac{\begin{array}{l} [c, \sigma \Vdash_R \varphi \text{ AU done}_= (b, \sigma')]_{\text{Rust}} \\ \left\{ \begin{array}{ll} [t, \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}, & \text{if } b \\ [u, \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}, & \text{otherwise} \end{array} \right. \end{array}}{[\text{if } c \{t\} \text{ else } \{u\}, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{IF}_{\text{RUST}}\text{AU}_L$$

Weakest precondition

$$\text{wp}(\text{if } c \{t\} \text{ else } \{u\}, \varphi \text{ AU } \varphi') = \text{wp}(c, \varphi \text{ AU done}_= (b, \sigma')) \wedge \begin{cases} \text{wp}(t, \varphi \text{ AU } \varphi')[\sigma'/\sigma], & \text{if } b \\ \text{wp}(u, \varphi \text{ AU } \varphi')[\sigma'/\sigma], & \text{otherwise} \end{cases}$$

Evaluating c may take multiple steps; φ must hold along the path. Uses $\varphi \text{ AU done}_=$ instead of $\text{AX done}_=$ because Rust conditions are effectful.

Rule: While Loop (AU — terminating)

Rust: `while c {t}` **with** invariant $\mathcal{R}$, decreases f

$$\frac{\begin{array}{c} \mathcal{R} \sigma \\ \forall \sigma, \mathcal{R} \sigma \rightarrow [c, \sigma \Vdash_R \varphi \text{ AU done} = (b, \sigma')]_{\text{Rust}} \\ \wedge \left\{ \begin{array}{l} [t, \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}} \vee \\ [t, \sigma' \Vdash_R \varphi \text{ AU AF done } (\lambda \sigma''. \mathcal{R} \sigma'' \wedge f \sigma'' < f \sigma)]_{\text{Rust}}, \text{ if } b \\ \varphi'(\sigma'), \text{ otherwise} \end{array} \right. \end{array}}{[\text{while } c \{t\}, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{WHILE}_{\text{RUST}} \text{AU}_L$$

Condition evaluation uses $\varphi \text{ AU done} =$ (multi-step, path-preserving). Direct lift of **While**_SAU_L with effectful conditions.

Rule: Loop (AG — infinite)

Rust: `loop {t}` **with** invariant $\mathcal{R}$ (**no** decreases)

$$\frac{\begin{array}{c} \mathcal{R} \sigma \\ \forall \sigma, \mathcal{R} \sigma \rightarrow \\ [\text{loop } \{t\}, \sigma \Vdash_L \varphi]_{\text{Rust}} \wedge \\ [t, \sigma \Vdash_R \varphi \text{ AU AF done } (\lambda \sigma' \Rightarrow \mathcal{R} \sigma')]_{\text{Rust}} \end{array}}{[\text{loop } \{t\}, \sigma \Vdash_L \text{AG } \varphi]_{\text{Rust}}} \text{LOOP}_{\text{RUST}} \text{AG}$$

Direct lift of **While**_SAG / **Iter**AG. Infinite loop = no exit condition. The coinductive hypothesis $[\text{loop } \{t\}, \sigma \Vdash_L \varphi]_{\text{Rust}}$ enables proving the property holds at the loop entry after each iteration.

Function Calls and Async/Await

Rule: Function Call (Bind Rule)

Rust: `let x = f(); k x`

$$\frac{\begin{array}{c} f \text{ ensures AF done } \mathcal{R} \\ \forall x, \sigma', \mathcal{R} \times \sigma' \rightarrow [k \ x, \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}} \end{array}}{[\text{let } x = f(); k \ x, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{CALL}_{\text{RUST}} \text{AU}_L$$

Contract-based transparent checking (Vera extension)

If f has temporal ensures Ψ , additionally assert $\Psi \implies \Phi$ where Φ is the caller's temporal obligation. This ensures f 's intermediate states satisfy the caller's properties.

Rule: Async Bind (Lazy Future Creation)

Rust: `let p = async f(); k p`

$$\frac{\text{No state change (lazy future)} \quad [k \ p, \ \sigma \Vdash_L \ \Phi]_{\text{Rust}}}{[\text{let } p = \text{async } f(); \ k \ p, \ \sigma \Vdash_L \ \Phi]_{\text{Rust}}} \text{ASYNCBIND}_{\text{RUST}}$$

Rust futures are lazy: `async fn()` returns a handle, body runs at `.await`. The temporal obligation Φ passes through to the continuation k unchanged.

Rule: Await (AU/AF — terminating callee)

Rust: `let x = p.await; k x`

$$\frac{\begin{array}{c} P[p] \text{ ensures } \varphi' \text{ AU AX done } \mathcal{R} \\ \varphi' \implies \varphi \text{ (transparent)} \\ \forall x, \sigma', \mathcal{R} \ x \ \sigma' \rightarrow [k \ x, \ \sigma' \Vdash_L \varphi \text{ AU } \varphi'']_{\text{Rust}} \end{array}}{[\text{let } x = p.\text{await}; k \ x, \ \sigma \Vdash_L \varphi \text{ AU } \varphi'']_{\text{Rust}}} \text{AWAIT}_{\text{RUST}} \text{AU}_L$$

- $P[p]$ is the async process behind future p
- The callee's path property φ' must imply the caller's φ (transparent await)
- After await returns with $\mathcal{R}$, continuation k continues the obligation

Rule: Await (AG — diverging callee)

Rust: $p.\text{await}$ where $P[p]$ ensures AG ψ

$$\frac{\begin{array}{c} P[p] \text{ ensures AG } \psi \\ \psi \implies \varphi \end{array}}{[p.\text{await}, \sigma \Vdash_L \text{AG } \varphi]_{\text{rust}}} \text{AWAIT}_{\text{RUST}} \text{AG}$$

- If the awaited process has AG ψ , it runs forever
- The await point becomes a *divergence point*
- The caller's AG φ is discharged if $\psi \implies \varphi$
- Continuation after await is unreachable (dead code)

Summary

wp(Rust, TICL) — Complete Rule Set

Rust Construct	TICL Rule	Imp Analog
<code>*x = e</code>	State update, check φ	Assign
<code>let x = t; u</code>	BindAU _L	Seq _S AU _L
<code>if c {t} else {u}</code>	Case split	If _S AU _L
<code>while c {t} + decreases</code>	IterAU _L	While _S AU _L
<code>loop {t} (no exit)</code>	IterAG	While _S AG
<code>f()</code>	Bind + transparent	BindAU _L
<code>let p = async f(); k</code>	No-op (lazy)	—
<code>p.await</code> (AU callee)	Bind + implication	BindAU _L
<code>p.await</code> (AG callee)	Divergence	—

Key extension over Imp: contract-based transparent temporal checking at function calls and `.await` boundaries.